AR Homework 3

Question (1)

(a) An Observable is the object that represents an event source and keeps track of subscribed callbacks. An Observer is one subscribed callback/listener. When the event occurs, the Observable notifies its Observers, and each Observer's callback runs.

(b) I would choose an aspheric lens because it can provide high image clarity with less spherical aberration and distortion than a simple spherical lens. A disadvantage is that aspheric lenses are more expensive and difficult to manufacture; they can also impose tighter alignment tolerances, making the headset more expensive and mechanically precise.

(c) I would use a web-based WebGL/WebXR workflow, for example Babylon.js in the browser. The main advantage is that users can open the demonstration directly from a link without installing an app application. A limitation is that performance and WebXR feature support depend on the browser and device, so it generally has less predictable performance and hardware access than a native application.

Date: / / /

(a) The default CYOS scene uses spherical ~~environment~~ environment mapping. First, the fragment position is transformed into view space and normalized into form e. The surface normal is also transformed into view space and normalized to form n. The shader then computes the reflected direction r = reflect(e, n). Since an environment map is a 2D image, this 3D reflection vector must be converted to 2D texture coordinates.

It ~~com~~ computes

$$m = 2\sqrt{r_x^2 + r_y^2 + (r_z+1)^2}$$

and then

$$vN = \frac{r_{xy}}{m} + 0.5$$

The division performs the spherical projection and the +0.5 shifts the coordinates into the [0,1] texture range. The shader samples refSampler at vN and uses that sampled RGB value as the fragment color. Therefore different surface normals sample different parts of the environment texture, producing the shiny reflective ~~effect~~ appearance of the torus knot.

(b) Clip-space coordinates put vertices into the coordinate system used for clipping and perspective projection. After the vertex shader computes clip coordinates, primitives outside the view volume can be clipped, and dividing by w converts the surviving vertices to normalized device

Date: / /

cordinates. This gives the GPU a standard representation independent of the original camera and world cordinates.

(f) A 3D model's surface is parameterized by 2D texture cordinates (u, v). Each surface vertex stores or interpolates a corresponding location in a 2D image, so the renderer can look up a color from the texture and place it on the 3D surface. The geometry is 3D, but the stored image only needs the 2D surface parameterization.

(g) Yes. A shear can be applied in the vertex shader by making one coordinate depend on another. For example, to shear x as a function of y:

p = position $\quad p_x = p_x + k \cdot p_y$

gl_Position = worldViewProjection · vec4(p, 1.0)

This corresponds to an x-shear matrix where x-y entry is k.

Question 3

(a) Using the assignment convention (camera looking along $-Z$ and pixel v increasing with NDC y), the projection matrix is:

Date: / / /

$$P = \begin{bmatrix} \frac{2f_x}{W} & 0 & 1 - \frac{2o_x}{W} & 0 \\ 0 & \frac{2f_y}{H} & 1 - \frac{2o_y}{H} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

(b) For $W = 800$, $H = 600$, $n = 0.2$, $f = 0.2$, $f_x = f_y = 800$, $o_x = 400$, $o_y = 400$:

$$P = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & \frac{8}{3} & -\frac{1}{3} & 0 \\ 0 & 0 & -\frac{101}{99} & -\frac{40}{99} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

(c) For $P_c = \begin{bmatrix} 0.3 \\ -0.15 \\ -6 \\ 1 \end{bmatrix}$,

$$PP = P_{clip} = \begin{bmatrix} x_{clip} \\ y_{clip} \\ z_{clip} \\ w_{clip} \end{bmatrix} = \begin{bmatrix} 2(0.3) \\ \frac{8}{3}(-0.15) \\ -\frac{101}{99}(-6) \\ -(-6) \end{bmatrix} = \begin{bmatrix} 0.6 \\ 1.6 \\ 57172 \\ 6 \end{bmatrix}$$

After dividing by 6 for the perspective:

$$P_{NDC} = \begin{bmatrix} 0.1 \\ 0.2667 \\ 0.9529 \end{bmatrix}$$

Date: / /

(a) Using

$$u = 0.5(x_{NDC} + 1)W$$
$$v = 0.5(y_{NDC} + 1)H$$

We get

$$u = 0.5(0.1 + 1)(800) = 0.5(1.1)(800) = 440,$$

and

$$v = 0.5(0.2667 + 1)(600) \approx 380$$

Thus, the pixel coordinates are $(u, v) = (440, 380)$.

Question 4

(b) The screenshot parameter changes are as follows:

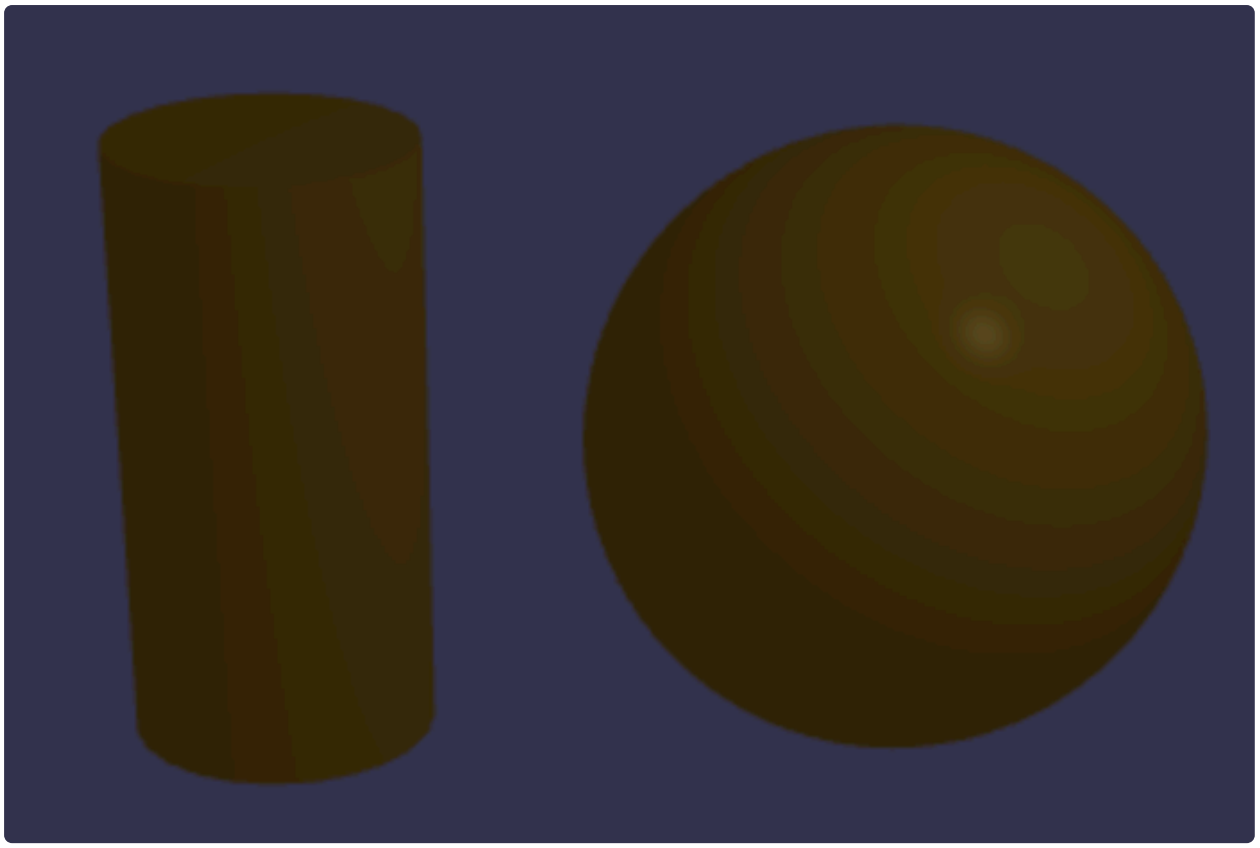
Screenshot 1: change uMatAmbient to (0.12, 0.02, 0.02) and uMatDiffuse to (0.80, 0.20, 0.20).

Screenshot 2: change uMatSpecular to (1.00, 1.00, 1.00) and uMatShininess to 120.0.

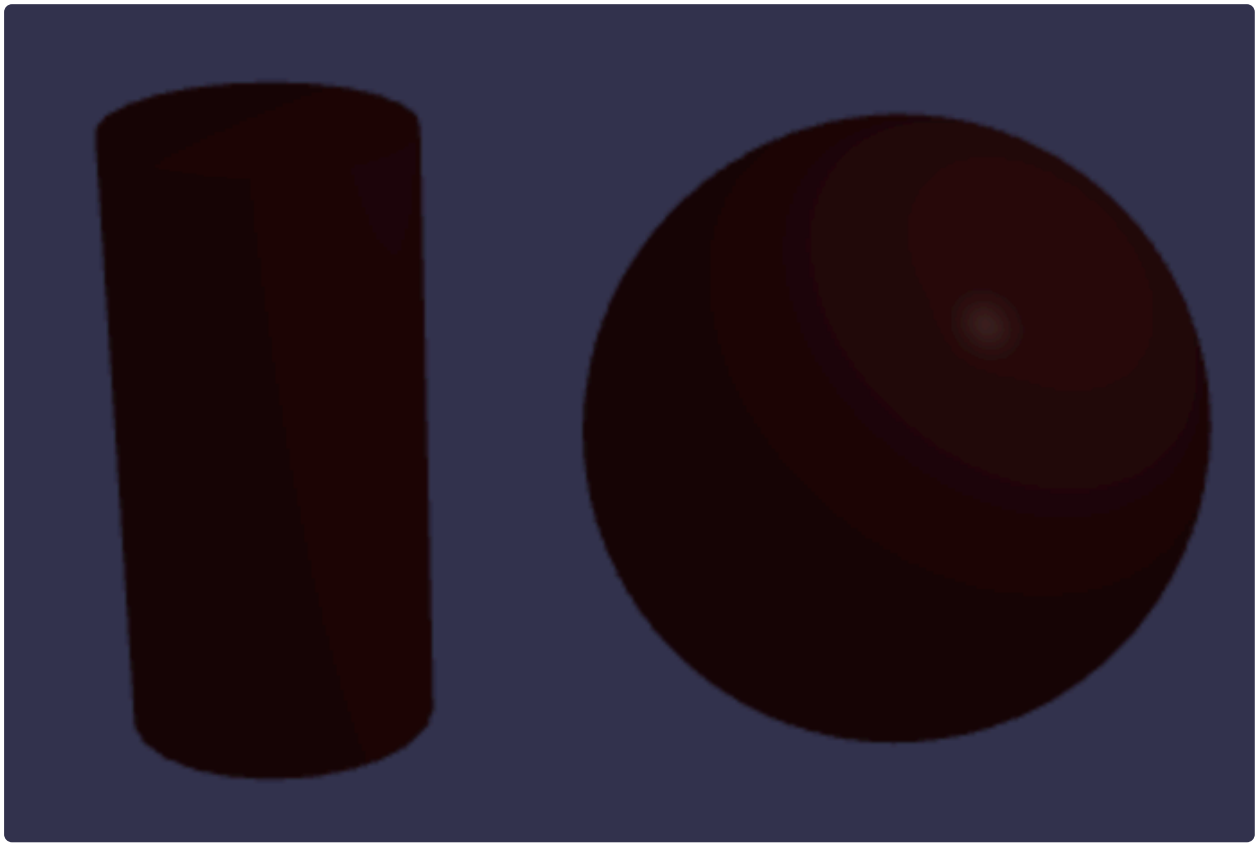
Screenshot 3: change uPointLightColor to (0.30, 0.60, 1.00) and uPointLightPos to (-3.0, 5.0, 1.0).

Intro AR HW3

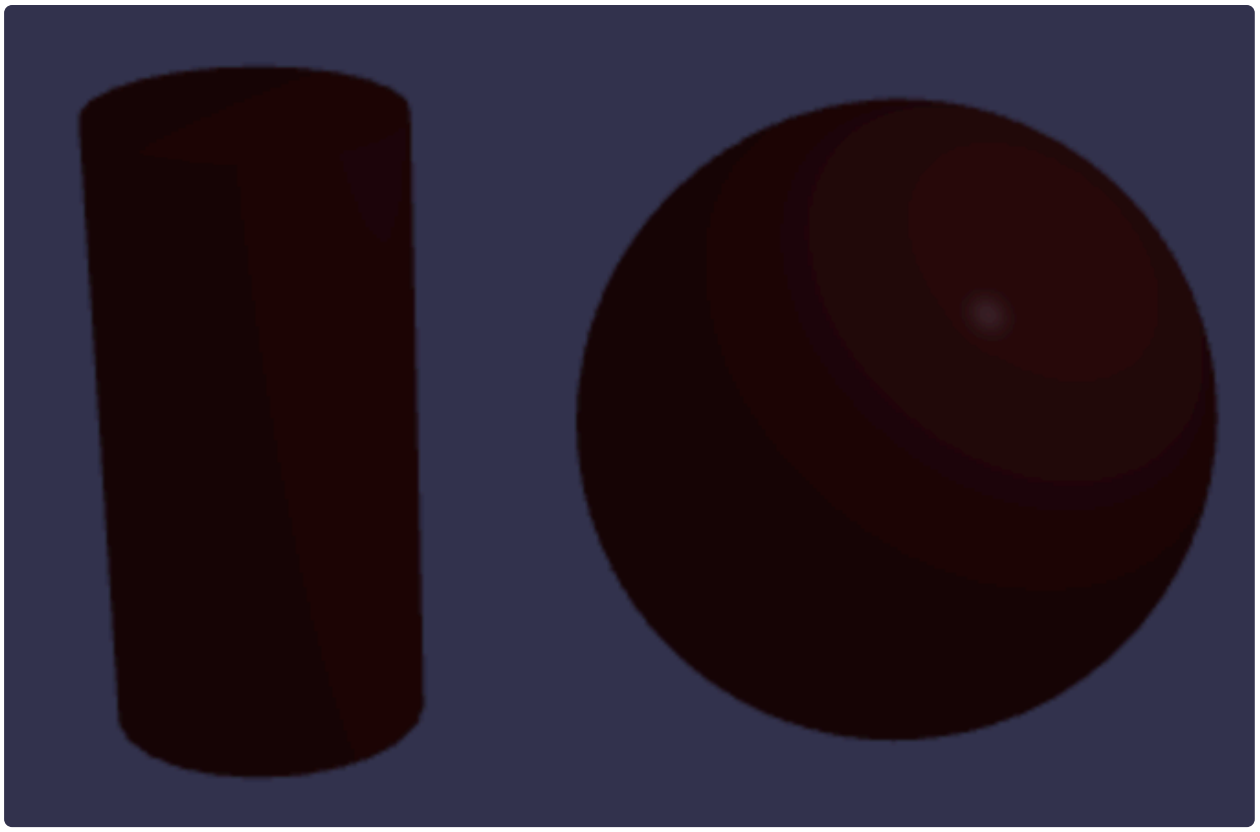
Q4a.



Q4b.



uMatAmbient is 0.12, uMatDiffuse is (0.80, 0.20, 0.20)



uMatSpecular is (1.00, 1.00, 1.00), uMatShininess is 128.0



uPointLightColor is (0.30, 0.60, 1.00), uPointLightPos(-3.0, 5.0, 1.0)

Q4c.

